

1 Hash function:

1.1 Perform the attack on the hash function based on the default files.

(Analysis -> Hash -> Attack on the hash value of the digital signature):

16- bit length using MD2:

Statistics of the Attack

Assumed efforts

Calculation time 0 year(s), 0 day(s), 0 hour(s), 0 minute(s) und 0.06 second(s)

Steps required 640

Efforts made to find a pair of messages

Calculation time 0 year(s), 0 day(s), 0 hour(s), 0 minute(s) und 0.06 second(s)

Steps required 943

Hash operations performed 2,390

Steps required sorted by run

Run ...	Steps until collision	Collision check	Total steps
1	504	439	943

Additional bytes

10 bytes were added to the harmless message.

10 bytes were added to the dangerous message.

Print statistics Cancel

16- bit length using md4:

Statistics of the Attack

Assumed efforts

Calculation time 0 year(s), 0 day(s), 0 hour(s), 0 minute(s) und 0.00 second(s)

Steps required 640

Efforts made to find a pair of messages

Calculation time 0 year(s), 0 day(s), 0 hour(s), 0 minute(s) und 0.00 second(s)

Steps required 1,468

Hash operations performed 3,820

Steps required sorted by run

Run ...	Steps until collision	Collision check	Total steps
1	270	239	509
2	614	345	959

Additional bytes

10 bytes were added to the harmless message.

10 bytes were added to the dangerous message.

Print statistics Cancel

Álvaro Puebla Ruisánchez / Óscar López García

CYBERSECURITY REPORT – LABORATORY 6

16-bit length using MD5:

Statistics of the Attack

Assumed efforts

Calculation time 0 year(s), 0 day(s), 0 hour(s), 0 minute(s) und 0.00 second(s)

Steps required 640

Efforts made to find a pair of messages

Calculation time 0 year(s), 0 day(s), 0 hour(s), 0 minute(s) und 0.00 second(s)

Steps required 823

Hash operations performed 2,060

Steps required sorted by run

Run ...	Steps until collision	Collision check	Total steps
1	414	409	823

Additional bytes

10 bytes were added to the harmless message.

10 bytes were added to the dangerous message.

Print statistics Cancel

16-bit length using SHA:

Statistics of the Attack

Assumed efforts

Calculation time 0 year(s), 0 day(s), 0 hour(s), 0 minute(s) und 0.00 second(s)

Steps required 640

Efforts made to find a pair of messages

Calculation time 0 year(s), 0 day(s), 0 hour(s), 0 minute(s) und 0.00 second(s)

Steps required 222

Hash operations performed 639

Steps required sorted by run

Run ...	Steps until collision	Collision check	Total steps
1	195	27	222

Additional bytes

10 bytes were added to the harmless message.

10 bytes were added to the dangerous message.

Print statistics Cancel

CYBERSECURITY REPORT – LABORATORY 6

16-bit length using SHA-1:

Statistics of the Attack

Assumed efforts

Calculation time: 0 year(s), 0 day(s), 0 hour(s), 0 minute(s) und 0.00 second(s)

Steps required: 640

Efforts made to find a pair of messages

Calculation time: 0 year(s), 0 day(s), 0 hour(s), 0 minute(s) und 0.00 second(s)

Steps required: 248

Hash operations performed: 696

Steps required sorted by run

Run ...	Steps until collision	Collision check	Total steps
1	200	48	248

Additional bytes

10 bytes were added to the harmless message.

10 bytes were added to the dangerous message.

Print statistics Cancel

1.2 Perform the attack again, but change the hash function and the significant bit length (at least two others):

32-bit length using MD2:

Statistics of the Attack

Assumed efforts

Calculation time: 0 year(s), 0 day(s), 0 hour(s), 0 minute(s) und 16.11 second(s)

Steps required: 163,840

Efforts made to find a pair of messages

Calculation time: 0 year(s), 0 day(s), 0 hour(s), 0 minute(s) und 2.33 second(s)

Steps required: 33,403

Hash operations performed: 83,784

Steps required sorted by run

Run ...	Steps until collision	Collision check	Total steps
1	16,978	16,425	33,403

Additional bytes

18 bytes were added to the harmless message.

18 bytes were added to the dangerous message.

Print statistics Cancel

CYBERSECURITY REPORT – LABORATORY 6

32-bit length using MD4:

Statistics of the Attack

Assumed efforts

Calculation time 0 year(s), 0 day(s), 0 hour(s), 0 minute(s) und 0.92 second(s)

Steps required 163,840

Efforts made to find a pair of messages

Calculation time 0 year(s), 0 day(s), 0 hour(s), 0 minute(s) und 0.40 second(s)

Steps required 400,824

Hash operations performed 1,022,710

Steps required sorted by run

Run ...	Steps until collision	Collision check	Total steps
1	182,270	142,450	324,720
2	38,792	37,312	76,104

Additional bytes

18 bytes were added to the harmless message.

18 bytes were added to the dangerous message.

Print statistics Cancel

32-bit length using MD5:

Statistics of the Attack

Assumed efforts

Calculation time 0 year(s), 0 day(s), 0 hour(s), 0 minute(s) und 1.06 second(s)

Steps required 163,840

Efforts made to find a pair of messages

Calculation time 0 year(s), 0 day(s), 0 hour(s), 0 minute(s) und 0.11 second(s)

Steps required 110,494

Hash operations performed 302,765

Steps required sorted by run

Run ...	Steps until collision	Collision check	Total steps
1	51,721	175	51,896
2	30,056	28,542	58,598

Additional bytes

18 bytes were added to the harmless message.

18 bytes were added to the dangerous message.

Print statistics Cancel

Álvaro Puebla Ruisánchez / Óscar López García

CYBERSECURITY REPORT – LABORATORY 6

32-bit length using SHA:

Statistics of the Attack

Assumed efforts

Calculation time 0 year(s), 0 day(s), 0 hour(s), 0 minute(s) und 1.92 second(s)

Steps required 163,840

Efforts made to find a pair of messages

Calculation time 0 year(s), 0 day(s), 0 hour(s), 0 minute(s) und 0.08 second(s)

Steps required 54,638

Hash operations performed 139,558

Steps required sorted by run

Run ...	Steps until collision	Collision check	Total steps
1	30,282	24,356	54,638

Additional bytes

18 bytes were added to the harmless message.

18 bytes were added to the dangerous message.

Print statistics Cancel

32-bit length using SHA-1:

Statistics of the Attack

Assumed efforts

Calculation time 0 year(s), 0 day(s), 0 hour(s), 0 minute(s) und 1.72 second(s)

Steps required 163,840

Efforts made to find a pair of messages

Calculation time 0 year(s), 0 day(s), 0 hour(s), 0 minute(s) und 0.18 second(s)

Steps required 193,692

Hash operations performed 497,297

Steps required sorted by run

Run ...	Steps until collision	Collision check	Total steps
1	67,821	51,369	119,190
2	42,092	32,410	74,502

Additional bytes

18 bytes were added to the harmless message.

18 bytes were added to the dangerous message.

Print statistics Cancel

CYBERSECURITY REPORT – LABORATORY 6

40-bit length using SHA:

Statistics of the Attack

Assumed efforts

Calculation time

0 year(s), 0 day(s), 0 hour(s), 0 minute(s) und 30.82 second(s)

Steps required

2,621,440

Efforts made to find a pair of messages

Calculation time

0 year(s), 0 day(s), 0 hour(s), 0 minute(s) und 3.10 second(s)

Steps required

2,120,918

Hash operations performed

5,617,976

Steps required sorted by run

Run ...	Steps until collision	Collision check	Total steps
1	1,376,140	744,778	2,120,918

Additional bytes

22 bytes were added to the harmless message.

22 bytes were added to the dangerous message.

Print statistics

Cancel

40-bit length using SHA-1:

Statistics of the Attack

Assumed efforts

Calculation time

0 year(s), 0 day(s), 0 hour(s), 0 minute(s) und 27.56 second(s)

Steps required

2,621,440

Efforts made to find a pair of messages

Calculation time

0 year(s), 0 day(s), 0 hour(s), 0 minute(s) und 2.50 second(s)

Steps required

2,392,546

Hash operations performed

7,140,342

Steps required sorted by run

Run ...	Steps until collision	Collision check	Total steps
1	2,355,250	37,296	2,392,546

Additional bytes

22 bytes were added to the harmless message.

22 bytes were added to the dangerous message.

Print statistics

Cancel

Álvaro Puebla Ruisánchez / Óscar López García

CYBERSECURITY REPORT – LABORATORY 6

40-bit length using MD2:

Statistics of the Attack

Assumed efforts

Calculation time 0 year(s), 0 day(s), 0 hour(s), 4 minute(s) und 17.85 second(s)

Steps required 2,621,440

Efforts made to find a pair of messages

Calculation time 0 year(s), 0 day(s), 0 hour(s), 4 minute(s) und 13.10 second(s)

Steps required 3,670,784

Hash operations performed 9,756,810

Steps required sorted by run

Run ...	Steps until collision	Collision check	Total steps
1	2,415,242	1,255,542	3,670,784

Additional bytes

22 bytes were added to the harmless message.

22 bytes were added to the dangerous message.

Print statistics Cancel

40-bit length using MD4:

Statistics of the Attack

Assumed efforts

Calculation time 0 year(s), 0 day(s), 0 hour(s), 0 minute(s) und 14.76 second(s)

Steps required 2,621,440

Efforts made to find a pair of messages

Calculation time 0 year(s), 0 day(s), 0 hour(s), 0 minute(s) und 3.95 second(s)

Steps required 4,543,461

Hash operations performed 11,370,066

Steps required sorted by run

Run ...	Steps until collision	Collision check	Total steps
1	2,283,144	2,260,317	4,543,461

Additional bytes

22 bytes were added to the harmless message.

22 bytes were added to the dangerous message.

Print statistics Cancel

CYBERSECURITY REPORT – LABORATORY 6

40-bit length using MD5:

The screenshot shows a window titled "Statistics of the Attack" with a close button (X) in the top right corner. It contains several sections of data:

- Assumed efforts:**
 - Calculation time: 0 year(s), 0 day(s), 0 hour(s), 0 minute(s) und 17.07 second(s)
 - Steps required: 2,621,440
- Efforts made to find a pair of messages:**
 - Calculation time: 0 year(s), 0 day(s), 0 hour(s), 0 minute(s) und 1.05 second(s)
 - Steps required: 1,119,603
 - Hash operations performed: 3,071,243
- Steps required sorted by run:**

Run ...	Steps until collision	Collision check	Total steps
1	832,037	287,566	1,119,603
- Additional bytes:**
 - 22 bytes were added to the harmless message.
 - 22 bytes were added to the dangerous message.

At the bottom, there are two buttons: "Print statistics" and "Cancel".

Here is a summary about the time it takes to crack the hashes:

	MD2	MD4	MD5	SHA	SHA-1
16-bit	0.06 s	0.00 s	0.00 s	0.00 s	0.00 s
32-bit	2.33 s	0.40 s	0.11 s	0.08 s	0.18 s
40-bit	4 min 13.10 s	3.95 s	1.05 s	3.10 s	2.50 s

1.3 How does changing the key length affect the time of the attack?

Increasing the number of significant bits increases the attack time exponentially. The expected effort is proportional to $2^{n/2}$; for example, increasing from 24 to 40 bits makes the attack roughly 256 times slower

1.4 Does the selection of the hash function affect the time of the collision search task (attack)?

Yes. The expected computational effort grows with the hash output length, but the actual wall-time depends on the function's per-hash speed and whether cryptanalytic attacks exist. Therefore choose a hash with sufficient digest length and no known practical collision attacks.

CYBERSECURITY REPORT – LABORATORY 6

1.5 What is the advantage of an attack (finding a collision) whose aim is to modify two documents than one?

Finding a collision between two documents is much more practical than attacking a single target because it is exponentially cheaper and allows signature replay/substitution, enabling a forger to obtain a valid signature on a benign file and transfer it to a malicious one without breaking the signer's key.

1.6 Can the hash function be regarded as safe? If so, for which parameters?

The hash function is secure only when a modern, unbroken algorithm such as SHA-2 or SHA-3 is used with a enough digest length (e.g., SHA-256 for 128-bit collision security or SHA-512 for 256-bit security). Shorter digests or obsolete hashes like MD5 and SHA-1 are no longer considered secure.

1.7 What types of problems have been identified for popular hash functions (MD5, SHA)?

Popular legacy hashes such as MD5 and SHA-1 suffer from practical collision and chosen-prefix attacks, reduced security margins, and structural weaknesses like length-extension. These flaws make them unsuitable for digital signatures or integrity verification; modern systems should use SHA-2 or SHA-3 instead.

CYBERSECURITY REPORT – LABORATORY 6

2.2.2 Text form:

➤ openssl pkeyparam -in dh_com_pub.pem -text

After sharing the same p and g from Kali to Ubuntu in any public media as p and g can be public:

```
(stud@kali-vm)-[~]
$ openssl pkeyparam -in dh_com_pub.pem -text
-----BEGIN DH PARAMETERS-----
MIIBDAKCAQEA4iHd5X6fuXCbfzRuRatx7ZdYSR3qbAb0qh9ya5XzG8kff0Gz0TH
A9zkJhW618vOF9Cp7Dai0L824SaGoCAiBuYiV1aUyA9FWTqL2bQ0SMY/ahFTEYg
570P065W77RLexEBHRvCDG/wKxqjeGengUoaL37v/3raunGANH7S9ER2PdB3ksB2
fqbzjyJz8UWQfKI5ebhi2H0Mty4VBIbff/ZgU96PsUG2xMs1PvfPy1bnb3xL476P
UJwrxz/gdWvJU7BAev9cCC6jWlOvZFM50K7HHM3v5pK8LIpbKBfo8MYZoqo+ZTqK
QQ/1bdqnfLFP02hGbpMKu0ZCH0mb33DwIBAgICA0E=
-----END DH PARAMETERS-----
DH Parameters: (2048 bit)
P:
00:e2:21:dd:e5:7e:9f:b9:70:81:7d:dc:d1:b9:16:
ad:c7:b6:5d:61:24:77:a9:b0:1b:3a:a8:7d:c9:ae:
57:cc:6f:24:7d:f3:86:cf:44:c7:03:dc:e4:26:15:
ba:d7:cb:ce:17:d0:a9:ec:36:a2:d0:bf:36:e1:26:
86:a0:20:22:06:e6:22:89:5d:5a:53:20:3d:15:64:
ea:97:66:d0:39:23:32:fd:a8:45:4c:46:20:e7:b3:
8f:d3:ae:56:ef:b4:4b:79:71:1b:1d:1b:c2:0c:6f:
f0:2b:1a:a3:78:67:a7:81:4a:1a:2f:7e:ef:ff:7a:
da:ba:71:80:34:7e:d2:f4:44:76:3d:d0:77:92:c0:
76:7e:a6:f3:8f:22:73:f1:45:90:7c:a2:39:79:b8:
62:d8:7d:0c:b7:2e:15:04:86:df:7f:f6:60:53:de:
8f:b1:41:b6:c4:cb:35:3e:f7:cf:cb:56:e7:6f:7c:
65:e3:be:8f:50:9c:2b:c7:3f:e0:75:6b:c9:53:b0:
40:7a:ff:5c:08:2e:a3:5a:53:af:64:53:12:d0:ae:
c7:1c:cd:ef:e6:92:bc:2c:8a:5b:28:17:e8:f0:c6:
19:a2:aa:3e:65:3a:8a:41:0f:f5:6d:da:a7:7e:51:
4d:3f:4d:a1:19:ba:4c:2a:ed:19:08:7d:34:99:bd:
f7:0f
G: 2 (0x2)
recommended-private-length: 225 bits

student@student-ubuntu:~$ openssl pkeyparam -in dh_com_pub.pem -text
-----BEGIN DH PARAMETERS-----
MIIBDAKCAQEA4iHd5X6fuXCbfzRuRatx7ZdYSR3qbAb0qh9ya5XzG8kff0Gz0TH
A9zkJhW618vOF9Cp7Dai0L824SaGoCAiBuYiV1aUyA9FWTqL2bQ0SMY/ahFTEYg
570P065W77RLexEBHRvCDG/wKxqjeGengUoaL37v/3raunGANH7S9ER2PdB3ksB2
fqbzjyJz8UWQfKI5ebhi2H0Mty4VBIbff/ZgU96PsUG2xMs1PvfPy1bnb3xL476P
UJwrxz/gdWvJU7BAev9cCC6jWlOvZFM50K7HHM3v5pK8LIpbKBfo8MYZoqo+ZTqK
QQ/1bdqnfLFP02hGbpMKu0ZCH0mb33DwIBAgICA0E=
-----END DH PARAMETERS-----
DH Parameters: (2048 bit)
P:
00:e2:21:dd:e5:7e:9f:b9:70:81:7d:dc:d1:b9:16:
ad:c7:b6:5d:61:24:77:a9:b0:1b:3a:a8:7d:c9:ae:
57:cc:6f:24:7d:f3:86:cf:44:c7:03:dc:e4:26:15:
ba:d7:cb:ce:17:d0:a9:ec:36:a2:d0:bf:36:e1:26:
86:a0:20:22:06:e6:22:89:5d:5a:53:20:3d:15:64:
ea:97:66:d0:39:23:32:fd:a8:45:4c:46:20:e7:b3:
8f:d3:ae:56:ef:b4:4b:79:71:1b:1d:1b:c2:0c:6f:
f0:2b:1a:a3:78:67:a7:81:4a:1a:2f:7e:ef:ff:7a:
da:ba:71:80:34:7e:d2:f4:44:76:3d:d0:77:92:c0:
76:7e:a6:f3:8f:22:73:f1:45:90:7c:a2:39:79:b8:
62:d8:7d:0c:b7:2e:15:04:86:df:7f:f6:60:53:de:
8f:b1:41:b6:c4:cb:35:3e:f7:cf:cb:56:e7:6f:7c:
65:e3:be:8f:50:9c:2b:c7:3f:e0:75:6b:c9:53:b0:
40:7a:ff:5c:08:2e:a3:5a:53:af:64:53:12:d0:ae:
c7:1c:cd:ef:e6:92:bc:2c:8a:5b:28:17:e8:f0:c6:
19:a2:aa:3e:65:3a:8a:41:0f:f5:6d:da:a7:7e:51:
4d:3f:4d:a1:19:ba:4c:2a:ed:19:08:7d:34:99:bd:
f7:0f
G: 2 (0x2)
recommended-private-length: 225 bits
```

Displays the parameters of a public key -> Specifically for Diffie-Hellman (DH) keys.

2.3 Based on common DH key, each client should generate their private keys:

➤ openssl genpkey -paramfile dh_com_pub.pem -out dh_client1_key.pem

Each part must create a private key (client1 -> a and client2 -> b) so they can share each public key given
 $p, g: A = g^a \pmod p$ for client 1 (Kali) and $B = g^b \pmod p$ for client 2 (Ubuntu)

```
(stud@kali-vm)-[~]
$ openssl genpkey -paramfile dh_com_pub.pem -out dh_client1_key.pem

student@student-ubuntu:~$ openssl genpkey -paramfile dh_com_pub.pem -out dh_client2_key.pem
```

CYBERSECURITY REPORT – LABORATORY 6

2.3.1 List the key:

- openssl pkey -in dh_client1_key.pem -text -noout

```
(stud@kali-vm)-[~]
$ openssl pkey -in dh_client1_key.pem -text -noout
DH Private-Key: (2048 bit)
private-key:
4f:73:a3:0b:fb:09:76:39:0a:d2:0d:99:d8:52:f5:
a4:0b:f6:85:00:cf:f3:3e:80:69:5b:8d:f6:3e:41:
39:8a:b1:6d:08:f5:6d:6c:82:9f:a3:2d:18:56:57:
9d:34:ab:ea:2e:82:30:e2:bf:d1:7e:d4:0c:c5:3d:
1c:d9:e3:43:71:89:4a:72:0f:68:91:ca:36:a3:e1:
29:98:33:fa:f0:79:06:06:8c:73:3b:61:81:e7:70:
88:8c:d1:1b:8c:64:f5:29:84:11:a2:9f:4b:da:5b:
5d:54:d3:09:0c:45:0d:80:5a:21:64:48:86:e2:1f:
84:d6:ea:74:53:3e:ea:e3:68:df:d9:2f:82:5a:09:
45:44:79:da:17:1c:23:1d:98:9e:93:b5:6a:ee:33:
4a:b3:41:89:87:c3:dd:08:17:5b:48:f5:93:34:d6:
01:6d:ee:07:40:1b:dc:85:a2:05:f3:05:20:95:57:
e2:61:9a:61:e2:dc:39:4f:36:28:41:4a:d4:47:60:
b1:79:0d:1c:fe:86:35:32:dd:f2:61:68:72:33:d9:
27:4e:a9:33:cd:77:f7:71:d1:9d:0b:d6:ca:c0:f0:
78:f7:6b:b4:f9:ef:75:e8:d3:a9:12:9e:b9:f6:5b:
42:1e:a3:41:13:65:70:ca:9c:7e:5d:cf:46:f4:73:
4f
public-key:
1e:b5:9e:53:a0:ce:89:16:59:00:05:16:f9:c1:84:
5e:69:35:16:3f:14:74:85:b4:91:7c:5d:c2:53:30:
17:d8:20:ef:e6:98:07:05:9a:0e:6e:3c:50:24:f8:
84:ab:34:3a:a7:23:94:d2:f7:9e:fd:8e:b4:8e:be:
38:35:69:9a:26:89:21:ef:6b:6a:72:dc:13:e6:a3:
98:e7:e0:eb:6b:1f:f0:c2:4e:70:d1:a9:90:34:ca:
6d:13:6b:ea:b9:d4:a2:a2:9e:71:07:e7:15:20:4a:
f6:12:ab:68:9c:a3:5f:07:a9:f4:91:fc:43:87:1e:
5b:aa:75:b5:d6:d1:87:72:1b:79:3f:d1:cb:63:b4:
43:a1:20:0d:57:c6:58:fb:22:44:b8:74:73:57:07:
69:f9:21:74:75:32:0d:64:fb:05:84:61:77:6b:7d:
1c:48:2c:c5:6c:a6:c3:7b:83:5d:e1:ba:de:77:d5:
14:39:a9:08:45:4a:7a:ff:b8:a2:3b:0a:b3:22:52:
05:46:a5:a3:74:5b:66:90:b0:05:fe:ec:42:17:71:
b6:ea:98:cf:52:41:9b:dc:3d:d4:3b:fb:89:d2:75:
1d:33:21:a8:e2:5d:e6:3a:d3:06:a9:1f:a1:54:a5:
3a:fa:c5:92:a8:d0:3e:d9:53:c4:b2:9f:c9:4f:1f:
b1
P:
00:e2:21:dd:e5:7e:9f:b9:70:81:7d:dc:d1:b9:16:
ad:c7:b6:5d:61:24:77:a9:b0:1b:3a:a8:7d:c9:ae:
57:cc:6f:24:7d:f3:86:cf:44:c7:03:dc:e4:26:15:
ba:d7:cf:ce:17:d0:a9:ec:36:a2:d0:bf:36:e1:26:
86:a0:20:22:06:e6:22:89:5d:5a:53:20:3d:15:64:
ea:97:66:d0:39:23:32:fd:a8:45:4c:46:20:e7:b3:
```

```
student@student-ubuntu:~$ openssl pkey -in dh_client2_key.pem -text -noout
DH Private-Key: (2048 bit)
private-key:
65:ed:9a:e7:49:73:0c:2b:f6:8a:fe:69:e8:2d:ac:
0d:7a:2e:f1:06:7d:d9:38:34:e8:8f:12:fe:98:0a:
c1:e9:77:76:d8:46:07:26:34:54:6a:63:b9:a4:52:
8b:d9:81:cc:24:15:96:17:6e:3a:37:6a:3f:5c:41:
03:78:79:29:4f:e4:d8:77:a7:58:a3:b0:e5:8b:f9:
26:20:2f:0b:94:5c:2b:9a:5c:25:0d:f1:73:f8:4f:
fb:86:75:f3:0e:29:30:fd:82:da:4f:cf:62:c1:d8:
2d:37:30:15:8d:b1:98:99:0a:d0:96:e9:21:c1:90:
cd:0e:16:f3:17:4c:54:e2:8c:a8:22:e6:6d:96:46:
47:6c:3d:bd:a3:c3:74:5a:77:f3:c4:28:05:40:b3:
26:56:ec:c1:a2:55:ac:c3:1a:84:1c:35:23:d6:1b:
6e:5c:7d:a5:83:7f:b7:83:ea:3e:a8:ab:1b:28:95:
da:61:c1:f0:35:b2:2b:03:e9:ed:96:a4:fe:19:12:
73:f6:22:e1:5b:56:c8:96:15:62:c2:98:b2:87:5c:
c9:e2:49:b6:85:80:12:8f:b4:ab:16:ce:92:29:50:
fe:48:71:55:b8:50:c6:11:ac:a2:7b:d3:45:87:4c:
cf:83:c3:d4:6a:1f:54:09:1a:be:43:d3:d3:18:24:
41
public-key:
56:44:dd:32:d3:43:6b:61:74:f6:bd:07:9f:17:a9:
eb:d5:85:23:fc:b9:20:bc:d3:38:78:5f:ab:5a:b5:
39:8a:ee:bd:47:40:86:94:4b:9c:8f:3c:d3:d1:90:
2c:f7:f7:e6:ad:ea:f5:11:d5:01:52:fd:5a:3b:79:
d7:0c:ee:1a:4f:96:6d:cf:76:4c:14:a4:1e:88:11:
0d:ef:05:10:ad:ef:77:62:58:57:91:eb:66:44:c7:
82:42:46:28:ab:1b:b5:99:38:3d:2e:7c:27:f4:6f:
ea:fd:ea:35:38:2f:68:14:3a:a0:2e:c5:ef:05:e8:
b9:1f:69:06:e0:94:a5:c8:1b:c7:58:d8:80:06:9f:
03:70:28:61:35:0a:78:98:5e:25:4f:26:1c:b0:f6:
7c:0f:47:c2:d6:2b:0c:c3:f6:af:79:30:4b:4c:ef:
```

A different public key/private key for every client.

2.4 Extract the public key based on the private key:

- openssl pkey -in dh_client1_key.pem -pubout -out dh_client1_pub_key.pem

```
(stud@kali-vm)-[~]
$ openssl pkey -in dh_client1_key.pem -pubout -out dh_client1_pub_key.pem
```

```
student@student-ubuntu:~$ openssl pkey -in dh_client2_key.pem -pubout -out dh_client2_pub_key.pem
```

CYBERSECURITY REPORT – LABORATORY 6

2.5 Exchange the public keys of the client between them and generate the shared secret key:

After sharing the public keys between client1 and client2:

- openssl pkeyutl -derive -inkey dh_client1_key.pem -peerkey dh_client2_pub_key.pem -out secret_key1.bin

```
(stud@kali-vm)-[~]
$ openssl pkeyutl -derive -inkey dh_client1_key.pem -peerkey dh_client2_pub_key.pem -out secret_key1.bin
```

```
student@student-ubuntu:~$ openssl pkeyutl -derive -inkey dh_client2_key.pem -peerkey dh_client1_pub_key.pem -out secret_key2.bin
```

2.6 Compare the shared secret keys using cmp command:

Only for testing purposes I exposed the client2 private key to kali just to check if they were equal:

```
(stud@kali-vm)-[~]
$ cmp secret_key1.bin secret_key2.bin
```

As no output is shown, the files are equal.

2.7 Create a hex dump of the shared secrets using xxd command:

```
(stud@kali-vm)-[~]
$ xxd -p secret_key1.bin > secret_key1.hex
```

```
student@student-ubuntu:~$ xxd -p secret_key2.bin > secret_key2.hex
```

Having both the same secret key of 2032 bits:

7e82e1f080e6a613764c2f1c2b38b642006e6195c0912ee0bda298759955

17c00c3291ba85dbb59d8cbb385c6140c834e0799588a9f155984b75bce8

b07cfd37724311e08ddf5e2087bbc394abd8e529a37996071618e1d59430

adac76c78d4f33e0de6af34677749d5d00e96c7b2b100d38272a0e2ca65c

e2983b972ff9efc14d2e3841beb0562c68d42d0c1744e0f6ea1898a9d447

9ff033ca6c9cea335757afc7649ee796064ac4e3c49d0ca4c75130215a1d

5eb01998c6b71ad605864d7d347386ca38626f9f7b37ae84ca97ac8adafe

3382cbd96013f06073bbb1a6a07781e1d6e420ede188d786a1fbb21026d5

6de958cafeb70532fee52c50dc20c035

CYBERSECURITY REPORT – LABORATORY 6

2.8 Which key exchange algorithm guarantees higher security? RSA versus DH. Why?

- **DH (Diffie–Hellman), when used ephemerally (DHE/ECDHE), is stronger in practice** because it provides **perfect forward secrecy**: compromise of a long-term private key does **not** reveal past session keys.
- **RSA key-transport** (static RSA encrypting a premaster secret) **does not provide forward secrecy**. If the RSA private key is later compromised, past session secrets can be recovered.
- Cryptographic hardness: RSA security relies on **integer factoring**, DH on the **discrete logarithm problem** (or ECDH on the elliptic-curve discrete log). For comparable bit-sizes, **ECDH gives more security per bit** than classical DH/RSA.
- Practical note: **both are broken by large quantum computers** (Shor’s algorithm), but for current classical adversaries ephemeral **DH is generally preferred**.

2.9 What are the risks for entities using DH key exchange protocol?

The **4 main problems** are:

- **Unauthenticated exchange (MitM)**: Without authentication, attackers can intercept and replace public keys, creating separate shared secrets with each victim.
- **Weak or malicious parameters**: Using small or attacker-chosen primes/generators makes the key mathematically breakable.
- **Poor randomness or key reuse**: Predictable or reused private values expose the shared secret and destroy forward secrecy.
- **Implementation flaws**: Timing leaks, invalid-group values, or side-channel attacks can reveal private exponents.

2.10 What are the other methods to determine a common cryptographic key?

- **Public-key encryption (RSA) key transport** -> sender encrypts a secret to receiver’s public key.
- **Password-Authenticated Key Exchange (PAKE)** -> SRP, J-PAKE, OPAQUE (secure password-based key agreement).
- **Key Distribution Center (KDC) protocols** -> e.g. **Kerberos**.
- **Pre-shared keys (PSK)** -> manual or out-of-band distribution.
- **Elliptic-curve variants of DH** -> **ECDH, X25519** (modern, efficient).
- **Hybrid KEX (TLS style)** -> combine public-key auth (certificates) + ephemeral DH for forward secrecy.
- **Physical exchange** (USB, QR, NFC) or **secure courier**.
- **Quantum Key Distribution (QKD)** (specialized hardware).
- After a shared secret is derived, use a **KDF** (HKDF etc.) to produce session keys.

CYBERSECURITY REPORT – LABORATORY 6

2.11 Which element of the DH protocol is not transferred between clients?

How does this affect security issues?

- **Not transferred:** each party's **private exponent** (the secret a or b) is **never sent**. Only the public values ($g^a \pmod p$, $g^b \pmod p$) are exchanged.
- **Security effect:** keeping private exponents secret prevents any eavesdropper who only sees $g^a \pmod p / g^b \pmod p$ from computing the shared key (assuming discrete log is hard). If a private exponent leaks or is reused, the shared key(s) are compromised and forward secrecy is lost. Also means attackers can only break the protocol by solving the discrete log or by MitM / replacing public values.

2.12 Check whether the last point has managed to establish the same key for both clients. Include the content of the agreed secret key in the report.

The shared private key is:

7e82e1f080e6a613764c2f1c2b38b642006e6195c0912ee0bda298759955

17c00c3291ba85dbb59d8cbb385c6140c834e0799588a9f155984b75bce8

b07cfd37724311e08ddf5e2087bbc394abd8e529a37996071618e1d59430

adac76c78d4f33e0de6af34677749d5d00e96c7b2b100d38272a0e2ca65c

e2983b972ff9efc14d2e3841beb0562c68d42d0c1744e0f6ea1898a9d447

9ff033ca6c9cea335757afc7649ee796064ac4e3c49d0ca4c75130215a1d

5eb01998c6b71ad605864d7d347386ca38626f9f7b37ae84ca97ac8adafe

3382cbd96013f06073bbb1a6a07781e1d6e420ede188d786a1fbb21026d5

6de958cafeb70532fee52c50dc20c035